Examination Management Portal (EMP)

Project report — Flask + SQLite web application · October 2026

Student	**[Your full name]**
Enrolment no.	**[Your enrolment number]**
Source code	Single ZIP with one root folder (`emp/`); run with `python app.py`
Demo video	**[Paste your video link here]** (also repeated on page 5)

1. Introduction

EMP is a role-based, server-rendered web application that manages oral/practical examinations end to end. An **Admin** creates courses and exams, defines a grading rubric and four date windows, assigns examiners and moves each exam through its lifecycle. **Examiners** (after Admin approval) publish time slots inside the slot-creation window and later score every rubric criterion for each student. **Students** register, search exams, book exactly one slot per exam inside the booking window, cancel before the deadline and finally see marks, a per-criterion breakdown and remarks once the Admin publishes results.

Core flow: Course → Exam (+ rubric, + examiners) → Slot Creation → Booking Open → Closed → evaluation → Completed → results published. Every action is a plain HTML form post or a link; JavaScript is used only for the optional charts on the Admin Reports page.

2. Technology and frameworks

Component	Version	Used for
Python	3.14	Language of the whole back-end
Flask	3.1.3	Web framework: routing, blueprints (`auth, admin, examiner, student`), sessions, flash messages
Flask-SQLAlchemy / SQLAlchemy	3.1.1 / 2.1.4	ORM: models, constraints, parameterised queries, transactions
SQLite	3 (built in)	Single-file database `instance/app.db`, created in code by `db.create_all()`
Werkzeug	3.1.9	Password hashing (`generate_password_hash` / `check_password_hash`), signed cookies
Jinja2	3.1.6	Server-side templates with template inheritance and auto-escaping
HTML5 + custom CSS	–	38 templates, one small stylesheet (`static/style.css`); no CSS framework, no front-end build step
Chart.js (CDN, optional)	4.4.1	Three charts on the Admin Reports page only

Only `Flask`, `Flask-SQLAlchemy` and `SQLAlchemy` are listed in `requirements.txt`; Werkzeug, Jinja2, itsdangerous, click and blinker are installed automatically as their dependencies.

3. How to run

```
pip install -r requirements.txt
python app.py            # creates instance/app.db and the Admin on first start
# open http://127.0.0.1:5000      Admin login: admin@emp.com / admin123
```

The Admin is seeded in code on first start; there is no Admin registration page. Students register and are active at once; Examiners register as *pending* and cannot use the portal until the Admin approves them.

4. Architecture

EMP is a single-process, server-rendered monolith. Routes stay thin: they read the form, call a service and render or redirect. Every business rule that matters lives in the service layer or in `helpers.py`, so a rule is written once and cannot be bypassed from a template.



Project structure

```
emp/
|-- app.py            app, blueprints, create_all, Admin seed
|-- extensions.py     db = SQLAlchemy()
|-- helpers.py        role_req, examiner_required,
|                     slot_creation_open, slots_editable,
|                     booking_open
|-- models/           one file per table + enums.py
|-- routes/           auth, admin, admin_exam,
|                     admin_extra, examiner, student
|-- services/         booking_service, evaluation_service,
|                     admin_service, profile_service
|-- templates/        base.html + admin/ examiner/
|                     student/ auth/ (38 files)
|-- static/style.css
|-- report/           this report
`-- requirements.txt
```

Security

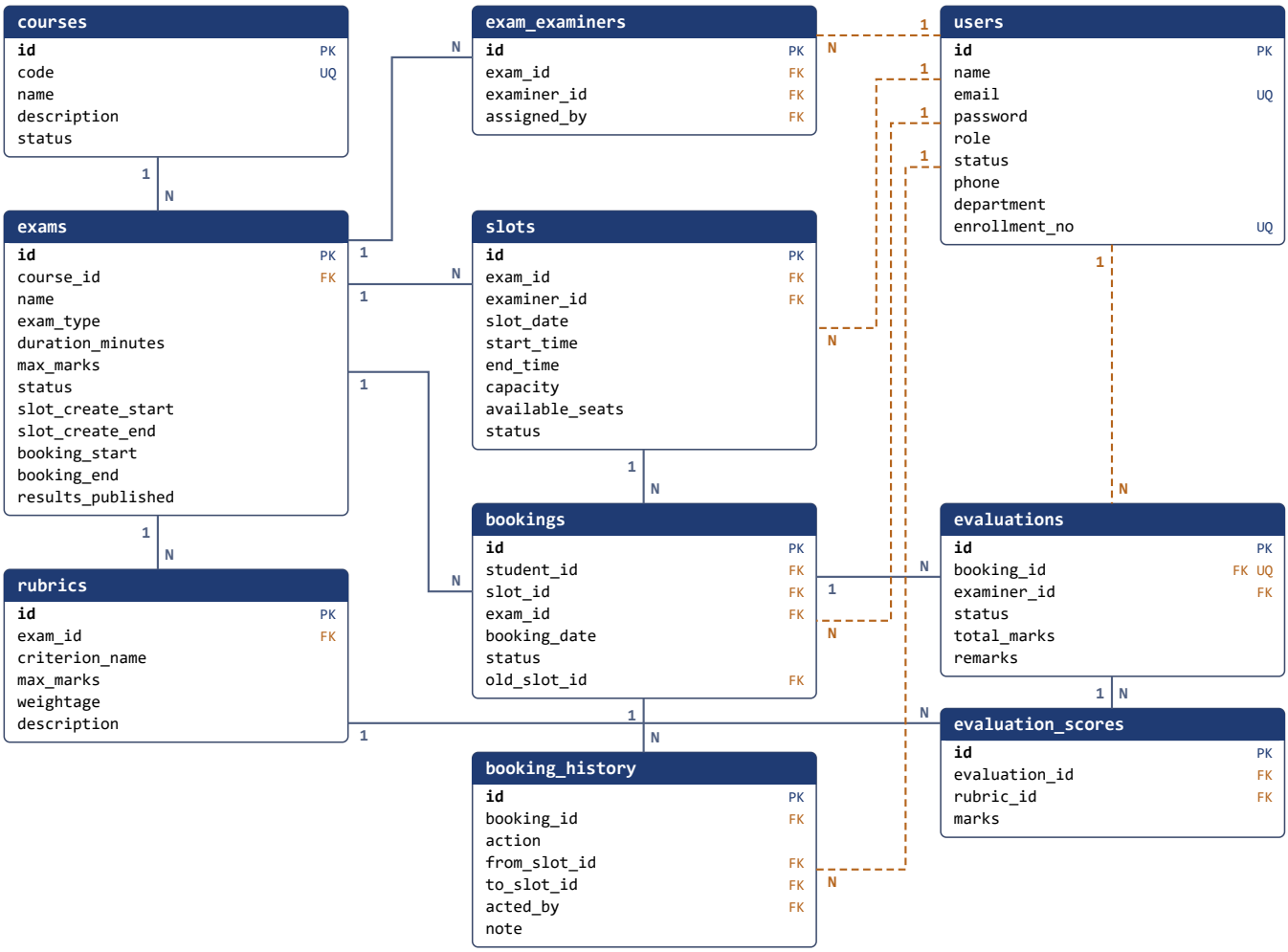
- Passwords stored only as Werkzeug hashes.
- Flask signed-cookie session holds `user_id`, `role`, `name`.
- `role_req(role)` guards each page: not logged in → login, wrong role → **403**.
- `examiner_required` also shows the "pending" page until the examiner is approved.
- Object-level checks: examiner → own slots/students only (403); student → own bookings/results only (404).
- All queries go through SQLAlchemy (parameterised); Jinja auto-escaping stays on.
- State-changing actions are POST forms, never links.
- Results are hidden (404) until the Admin publishes them.

5. Business rules and where they are enforced

Rule	Enforced by
Slots only in the creation period	`slot_creation_open`: status Slot Creation **and** now inside the date window; examiner must be assigned
Edit / cancel slot before booking starts	`slots_editable` (before `booking_start`) and no active bookings in the slot
Book only in the booking period	`booking_open` inside `booking_service.book`
No overbooking	`UPDATE slots SET available_seats = available_seats - 1 WHERE id=? AND available_seats > 0`; 0 rows changed = full. Plus `CHECK(available_seats >= 0 AND <= capacity)`
Slot Full at 0 seats, reopened on cancel	status flips Available ↔ Full in the same transaction
One booking per exam	service pre-check + partial unique index on `(student_id, exam_id) WHERE status='booked'`
No overlapping bookings (extra)	`_clashes`: same date and overlapping times
Cancel before the deadline	status Booked and now < `booking_end`; seat returned
Examiner evaluates only own students	`slot.examiner_id == session user`, otherwise 403
Marks formula	total = Σ(awarded ÷ criterion max × weightage) ÷ 100 × exam max marks
Exam lifecycle	Draft → Slot Creation (needs weightage = 100 and ≥1 examiner) → Booking Open → Closed → Completed (needs every booking evaluated) → publish
Full history, never deleted	`booking_history` rows for book, cancel, reschedule, examiner change, completion
Admin reschedule / change examiner	`admin_service`: one transaction, free seat, no clash, approved examiner; logged
Deactivate examiner	refused while they own upcoming slots with bookings

6. Database design (ER diagram)

PK primary key · **FK** foreign key · **UQ** unique. Solid lines: parent → child inside the exam data. Dashed brown lines: references to `users` (examiner / student / acting admin). **1** = one, **N** = many. Every table also has a `created_at` column.



Relationships

- One **course** has many **exams**.
- One **exam** has many rubric criteria, slots, bookings and assigned examiners (`exam_examiners`, unique per exam + examiner).
- One **user** with role examiner owns many slots; a user with role student has many bookings.
- One **slot** has many bookings; `bookings.old_slot_id` remembers a reschedule.
- One **booking** has many `booking_history` rows and at most one **evaluation** (`booking_id` unique).
- One **evaluation** has one `evaluation_scores` row per rubric criterion (unique per evaluation + rubric).
- Roles (`admin, examiner, student`) share one `users` table; `status` is active / pending / rejected / inactive.
- Enums are stored as lowercase text; CHECK constraints guard marks, capacity, seats, time and date order.

Status lifecycles (stored in the `status` columns)

Entity	Allowed values and moves
Exam	draft → slotcreation → bookingopen → closed → completed; `results_published` (0/1) is set after completed. Each step is an Admin button, and the date windows must also hold.
Slot	available ↔ full as seats change; cancelled when the examiner removes it; completed when the exam is completed.
Booking	booked → cancelled (student or Admin) or booked → completed (after the examiner submits the evaluation).
Evaluation	draft → submitted. Results are visible only when it is submitted **and** the exam's results are published.
User	students: active. examiners: pending → active or rejected; active ↔ inactive. Admin: active (seeded).

7. Endpoint table (57 routes)

Generated from the running Flask app. Every page after login lives under its role prefix (`/admin`, `/examiner`, `/student`); opening another role's URL returns **403**. POST routes are plain HTML form submissions followed by a redirect. The optional JSON API (`/api/v1`) is not part of this build.

Method	Route	Access	Purpose
GET	`/`	Public	Landing page
GET/POST	`/login`	Public	Login for all roles, redirects by role
GET/POST	`/register/<role>`	Public	Register as student / examiner (examiner starts pending)
POST	`/logout`	Any user	End the session
GET	`/admin/bookings`	Admin	All bookings with filters
GET	`/admin/bookings/<booking_id>`	Admin	Booking detail and history
POST	`/admin/bookings/<booking_id>/cancel`	Admin	Cancel a booking
POST	`/admin/bookings/<booking_id>/reschedule`	Admin	Move a booking to another slot
GET	`/admin/bookings/export.csv`	Admin	Download bookings as CSV
GET	`/admin/courses`	Admin	Course list
POST	`/admin/courses/<course_id>/delete`	Admin	Delete a course (blocked if it has exams)
GET/POST	`/admin/courses/<course_id>/edit`	Admin	Edit a course
POST	`/admin/courses/<course_id>/toggle`	Admin	Activate / deactivate a course
GET/POST	`/admin/courses/new`	Admin	Create a course
GET	`/admin/dashboard`	Admin	Totals and pending-examiner count
GET	`/admin/examiners`	Admin	Examiner list and approval queue
GET	`/admin/examiners/<user_id>`	Admin	Examiner profile and workload
POST	`/admin/examiners/<user_id>/<action>`	Admin	Approve / reject / deactivate / reactivate
GET	`/admin/exams`	Admin	Exam list
GET	`/admin/exams/<exam_id>`	Admin	Exam page: rubric, examiners, status buttons
POST	`/admin/exams/<exam_id>/delete`	Admin	Delete an exam (Draft only)
GET/POST	`/admin/exams/<exam_id>/edit`	Admin	Edit an exam (Draft only)
POST	`/admin/exams/<exam_id>/examiners/<user_id>/remove`	Admin	Remove an assigned examiner
POST	`/admin/exams/<exam_id>/examiners/add`	Admin	Assign an examiner to an exam
POST	`/admin/exams/<exam_id>/rubrics/<rubric_id>/delete`	Admin	Remove a rubric criterion
POST	`/admin/exams/<exam_id>/rubrics/new`	Admin	Add a rubric criterion
POST	`/admin/exams/<exam_id>/status/<action>`	Admin	Lifecycle step / publish results
GET/POST	`/admin/exams/new`	Admin	Create an exam with its four dates
GET	`/admin/history`	Admin	Completed exams, filter by course / date
GET	`/admin/reports`	Admin	Charts: trend, workload, completion
GET	`/admin/search`	Admin	Search students, examiners, exams, bookings
GET	`/admin/slots`	Admin	All slots with filters
GET	`/admin/slots/<slot_id>`	Admin	Slot detail and its students
POST	`/admin/slots/<slot_id>/change-examiner`	Admin	Change the examiner of a slot
GET	`/admin/students`	Admin	Student list with search
GET	`/admin/students/<user_id>`	Admin	Student profile, bookings, activity log
GET	`/examiner/dashboard`	Examiner	Assigned exams, slots, pending evaluations
GET/POST	`/examiner/evaluate/<booking_id>`	Examiner	Score every criterion + remarks
GET	`/examiner/evaluations`	Examiner	Pending and finished evaluations
GET	`/examiner/exams`	Examiner	Assigned exams
GET	`/examiner/exams/<exam_id>`	Examiner	Exam, rubric and own slots
GET/POST	`/examiner/profile`	Examiner	Edit contact details and password
GET	`/examiner/slots`	Examiner	My slots
POST	`/examiner/slots/<slot_id>/cancel`	Examiner	Cancel a slot (before booking, no bookings)
GET/POST	`/examiner/slots/<slot_id>/edit`	Examiner	Edit a slot (before booking, no bookings)
GET	`/examiner/slots/<slot_id>/students`	Examiner	Roster of one slot
GET/POST	`/examiner/slots/new`	Examiner	Create a slot (creation window only)
GET	`/student/bookings`	Student	My active bookings
POST	`/student/bookings/<booking_id>/cancel`	Student	Cancel before the booking deadline
GET	`/student/dashboard`	Student	Open exams and upcoming bookings
GET	`/student/exams`	Student	Search / filter exams
GET	`/student/exams/<exam_id>`	Student	Exam with slots and seats left
GET	`/student/history`	Student	All bookings and activity log
GET/POST	`/student/profile`	Student	Edit profile and password
GET	`/student/results`	Student	Published results
GET	`/student/results/<booking_id>`	Student	Marks per criterion and remarks
POST	`/student/slots/<slot_id>/book`	Student	Book a slot (all guards applied)

8. Testing

- **Manual walkthrough** of the full story: register examiner (blocked) → Admin approves → course, exam, rubric, assignment → slots → two students book, the third is refused at capacity → evaluation → complete → publish → student sees results.
- **Scripted request-level checks** (79 assertions, all passing): every page for every role, booking and seat counts, duplicate / full / clash refusals, cancel returns the seat, evaluation maths and range checks, 403 / 404 access rules, reschedule, change examiner, deactivation rule, profile and password changes.
- **Clean-install check:** in a brand-new virtual environment `pip install -r requirements.txt` followed by `python app.py` creates the database and the Admin and serves every page.

9. Known limitations and future work

- CSRF tokens (Flask-WTF) are not implemented yet; the secret key and default Admin password are development values and must be changed before real use.
- The optional JSON API, PDF scorecard and e-mail notifications are not implemented. Lists are not paginated.
- SQLite allows one writer at a time (fine for a local demo); times are stored as local time without a time-zone.

10. AI / LLM declaration

AI assistance was used in this project. The large-language model **Claude (Anthropic)**, used through the Claude Code tool, helped with: drafting the architecture and database design, writing and reviewing code (routes, services, models, templates, CSS), debugging error messages, writing test checks, and writing the README and this report. I ran the application, tested the flows, read the code that was produced and can explain it. All AI-generated content was reviewed by me before submission.

11. Frameworks and libraries used

Flask 3.1.3 · Flask-SQLAlchemy 3.1.1 · SQLAlchemy 2.1.4 · Werkzeug 3.1.9 · Jinja2 3.1.6 · itsdangerous 2.2.0 · click 8.5.0 · blinker 1.9.0 · MarkupSafe 3.0.4 · SQLite 3 · Chart.js 4.4.1 (CDN, optional) · HTML5 / CSS3 (no CSS framework).

12. Demo video

Video link	[Paste your video link here]
Suggested demo order	Admin login → approve examiner → course / exam / rubric / assign → examiner slots → student books (show full slot) → evaluation → publish → student results → Admin reschedule, change examiner, search, reports